

Trabajo practico 1

De Algoritmos y estructura de datos

Jazmin Oliverio

111787

joliverio@fi.uba.ar

El trabajo realizado se trata de el desarrollo de un juego en consola basado en texto, donde el jugador se mueve en un tablero donde debe buscar enemigos, enfrentarse a ellos y debe derrotarlos para ganar la partida. El objetivo principal de este trabajo es aplicar los conceptos básicos del idioma como bases, objetos, atributos, métodos, modularización y estructura de control.

Informe del trabajo realizado.

Desarrollo.

Comencé creando las clases básicas de el juego como **Menú**, **Jugador**, **Enemigo**, que fueron relativamente rápidas ya que solo habia que declarar atributos, y ademas yo sabia que mas me iba a llevar tiempo el combate o como imprimir el tablero.

Luego me di cuenta que iba a necesitar mas clases para poder organizar mejor el personaje y los enemigos, por eso cree las dos clases llamadas **ControlJugadores** y **ControlEnemigos**, estas clases me llevaron un poco mas de tiempo que las anteriores ya que me costo comprender como hacer para generar enemigos de forma aleatoria a lo largo de todo el tablero, pero igualmente una vez que comprendí como usar la librería Random se me facilito mucho mas, también, en ControlJugadores, agregue los movimientos permitidos por el personaje a lo largo de la partida, esto tranquilamente lo podría haber creado en la clase Tablero, pero cuando hice el trabajo me pareció mas coherente aquí ya que los movimientos son del jugador.

En la clase **Tablero** me trabe al intentar imprimir solo una parte de el. El foco central debía ser el personaje pero que solo muestre un tablero de 3x3 a su alrededor fue Aldo que me costo. Comencé imprimiendo todo el tablero, de 10x10 completo, cuando, en realidad las únicas posiciones que me importaban eran las de su alrededor, esto hacia que pareciera que estaba imprimiendo un tablero chiquito dentro de uno mas grande. Luego de informarme y pensarlo bastante me di cuenta que podría probar con limitar los rangos de impresión, de esta forma, aunque el tablero sea de 10x10 en memoria el usuario solo vería una parte de este.

El **Combate** fue algo que me llevo bastante tiempo de pensar, lo que yo quería que mi juego hiciera pero también lo mas divertido, quería crear una forma un poco mas dinámica con el usuario, sin que se vuelva muy complejo, por eso opte por que haya dos opciones, una donde el jugador ataque con un riesgo bajo y también haga un ataque con un riesgo alto, que ambos ataques y riesgos sean aleatorios me parecían divertidos también para la experiencia del usuario.

En la clase de **Juego**, que fue la ultima que termine de acomodar ya que es ahi donde mi juego tomo forma, fue algo que lo fui haciendo a medida que avanzaba con el resto del código.

Igualmente, creo que la parte mas divertida de todo esto fue probarlo y divertirme jugándolo.

Cuestionario.

Responder el siguiente Cuestionario:

1) ¿Qué es un Debug?

El debug es el proceso de encontrar y corregir errores del programa. Viene de la palabra bug que en ingles significa 'bicho', podría relacionarse como algo que no debería estar ahí y que molesta a la hora de lograr un funcionamiento correcto.

Se trata de ejecutar el código e ir recorriendo linea por linea para asi poder encontrar el error de forma mas sencilla, observando cómo cambian las variables y qué instrucciones se ejecutan.

2) ¿Qué es un "Breakpoint"?

Un breakpoint, o un punto de quiebre, traducido del ingles es como un stop que el programador le puede colocar al código para que la ejecución se detenga ahí. Se usa a la hora de debuguear un programa ya que sirve para analizar el programa justo antes o después de ese stop.

3) ¿Qué es "Step Into", "Step Over" y "Step Out"?

- Step into viene de "ingresar" y significa que si hay una llamada a una función, va adentro de la función y puedes ver como esta se ejecuta linea por linea.
- Step over lo podemos traducir a un "omitir" o "evitar" y lo que hace es ejecutar la linea actual completa pero sin entrar a la función, evitandola, y pasando a la siguiente linea.
- Step out puede traducirse como un "salir" del ingles, termina de ejecutar el método actual y vuelve al punto donde fue llamado.

Manual del usuario.

Introducción

Este juego es un trabajo práctico donde el jugador se mueve en un tablero, se encuentra con enemigos y tiene que derrotarlos para ganar. Todo se juega desde la consola y se maneja con el teclado. Es bastante sencillo: tenés que moverte, pelear y tratar de sobrevivir hasta eliminar a los tres enemigos.

Requisitos

Para poder jugar este juego lo único que se necesita es:

- Una computadora.
- Tener instalado **Java**.

Cómo ejecutar el juego

1. Abrir la consola.
2. Ir a la carpeta donde están los archivos del juego.
3. Ejecutar la clase principal con el comando: *java Tp1.Juego*
4. Con eso el juego ya empieza a correr.

Cómo se juega

1. Al inicio, aparece un menú con un mensaje de bienvenida y unas instrucciones básicas.
2. Lo primero que se pide es ingresar el nombre del personaje. Ese nombre va a acompañar al jugador durante toda la partida.
3. Una vez ingresado el nombre, se imprime un tablero de 3x3, donde el personaje aparece siempre en el centro.
4. Para moverse dentro del tablero se usan las teclas:
 - **W** → arriba
 - **S** → abajo
 - **A** → izquierda
 - **D** → derecha
5. El objetivo es buscar a los enemigos, que están repartidos de forma aleatoria en el tablero.

6. Cuando el jugador se para en la misma casilla que un enemigo, empieza automáticamente un combate.

El combate

- Cuando comienza un combate, aparece un pequeño menú con opciones de ataque:
 - Opción 1: Ataque rápido → menos daño, menos riesgo.
 - Opción 2: Ataque fuerte → más daño, pero con más riesgo.
- Cada ataque genera un daño aleatorio tanto al enemigo como al jugador.
- La pelea sigue hasta que uno de los dos se quede sin vida.
- Si gana el enemigo, la partida termina.
- Si gana el jugador, puede seguir moviéndose por el tablero para buscar al siguiente enemigo.

Recuperación de vida

Entre un combate y otro, cuando el jugador se mueve por el tablero, va recuperando un pequeño porcentaje de la vida actual en cada turno. Esto hace que, aunque salga lastimado de una pelea, tenga más chances de recuperarse antes de la siguiente.

Cómo ganar o perder

- El jugador gana si logra derrotar a los tres enemigos.
- El jugador pierde si su vida llega a 0 en cualquier combate.
- Al final de la partida (sea ganando o perdiendo), el juego muestra en pantalla el tiempo total que duró la partida.

Manual del programador.

Comencé creando las clases básicas de el juego como **Jugador**, donde primero defino los atributos del personaje, muestro la información de cada jugador y cree un método donde el personaje recupera un porcentaje de vida por cada movimiento que hace dependiendo de la cantidad de vida que tenga el personaje, sin superar la vida maxima(100).

Luego seguí por la clase **Enemigo**, donde primero defino los atributos del enemigo y muestro la información de cada enemigo.

La siguiente clase fue **Tablero**, donde inicializo las columnas y las filas, creamos el tablero de NxM, llenamos las casillas vacías del tablero por puntos, colocamos los enemigos y el jugador en el centro del tablero e imprimimos una parte del tablero, ya que el tablero entero es de 10x10 pero mostramos un tablero de 3x3.

La clase **Menú** muestra las instrucciones y solicita el ingreso del nombre del jugador.

Luego tenemos dos clases donde manejamos el control de el personaje y de los enemigos:

- En la clase **ControlEnemigos** utilizamos un método para generar una cantidad especifica de enemigos y colocarlos de forma aleatoria en el tablero con la librería Random de java.
- La clase **ControlJugadores** comienza colocando el jugador en el centro del tablero mediante las posiciones en el eje X e Y, ademas también en esta clase se encuentra el método que mediante la letra ingresada por el usuario va moviendo el personaje a lo largo de el tablero para poder encontrar los enemigos.

La clase **Combate** maneja la pelea entre el jugador y el enemigo, una vez que el programa comprende que el personaje encontró un enemigo ofrece dos tipos de ataques, donde puede ser un ataque fuerte o uno un poco mas leve, menos dañino, dependiendo de la opción que elija el usuario, el personaje va a atacar de forma aleatoria, en ambos casos, ya que utilice la librería Random

otra vez para poder generar todos combates aleatorios a lo largo de la partida. Luego muestra el estado y el resultado final de la partida.

La ultima clase de el trabajo es la clase **Juego**, considerada como el main, comenzamos guardando la hora de inicio de partida mediante la librería Times de java, luego muestra el menú y recibe el nombre del jugador. Una vez que el usuario brinda esa información el programa crea un tablero de 10x10, igualmente el programa imprime un tablero de 3x3, ya que simula un tablero infinito visualmente, también crea un personaje, señalado con la letra P, que va a estar ubicado siempre en el centro de esta grilla. Genera un control de movimiento y crea 3 enemigos que se encuentran dispersados en toda la grilla de forma aleatoria, esto permite que en cada partida los enemigos se encuentren en un lugar siempre diferente.

Luego inicializa el combate y va verificando, a medida que el personaje se vaya moviendo si este cae arriba de un enemigo, de ser asi, verifica si el combate termino en una derrota o una victoria e imprime el resultado final, luego de esto corta y guarda el tiempo final de partida e imprime los segundos entre el tiempo inicial y el tiempo final de este.